

[Skip to main content](#) [r/WebRTC](#) [Search in r/WebRTC](#)

Create

**r/WebRTC** • 6mo ago
Solid-Band3204

...

Scaling Mediasoup SFU horizontally for N:N (up to 20 people per room), audio sharing only

I've been exploring WebRTC related systems for a few weeks, and I find them quite interesting. My question is about scaling WebRTC systems. When scaling WebRTC in a **P2P** setup, we typically just scale the **signaling server**. If signaling is done through WebSocket, we can use something like **Redis** or another **pub/sub server** to pass the signaling messages between servers. That way, we can horizontally scale the P2P WebRTC system that's what I've learned so far. However, things get confusing when it comes to **SFU** architecture. SFUs also use WebSocket for signaling, but unlike P2P, in SFU setups we need a **persistent WebSocket connection** between clients and the SFU.

In P2P, after signaling is complete, peers communicate directly and if NAT traversal fails through STUN, it's handled by a TURN server. But in the SFU case, since media always passes through the SFU, I'm not sure how scaling works.

Let's say I'm running one SFU worker on one server instance, and all my routers depend on that worker. When this worker becomes overloaded, I'd like to spin up another server instance and use the same pub/sub signaling setup as before. But how do they communicate with each other across different SFU instances through the pub/sub system? This part really confuses me

Can anyone help me understand how to **horizontally scale an SFU (Mediasoup)** properly?
also tell me guys if I have any wrong understanding of anything



5



7

[Join the conversation](#)Sort by: **Best** ▾

Search Comments

**Patm290** • 6mo ago

Some basics to start with:

1. Media resides in the server you produced (sent) the media to; only way to get media sent to Server1 in Server2 is to use pipe transports; I am assuming different machines here. Alternatively, you could produce (send) to both (all) servers from the client (producer) side; not ideal.
2. Media can be consumed from where the media is available, if Person1 media is on Server1, only Server1 connections can receive(consume) it; it means if you have others on Server2 to ServerN that need the same media of Person1, you pipe there as indicated in the first point.
3. Proceed from there in tracking the numbers per server and limits you impose to avoid having CPU and most importantly bandwidth being bottlenecks; quick checks on number of transports (transport carries media) you can max at, can help

[Skip to main content](#)[+ Create](#)

5. Gets more tricky for large number of concurrent producers in same room, say 1000+ people actively producing media; figure out things like dedicated producing and consuming endpoints and the like; or use a cost-effective cloud option like **MediaSFU**

3 [Reply](#) ...



Solid-Band3204 OP • 6mo ago

thanks for your feedback now i get kind of idea how this things are scale altho they are not that much simple that i am initlay tinkng also i just read little bit about pipe transports and they are expensive task to do i need use this ting properly this adds up more complexcity

1 [Reply](#) ...



Patm290 • 6mo ago

Note: You may not even need to get to the level of pipe transports if it seems daunting to you.

Assuming you are maxing at 20 per room (unless you have more people than that), you make sure all 20 are assigned to the same server. Your task now is just keeping note of rooms available in specific servers and routing consumers there. In that way no consumer lands in a server where expected media is not available.

Once again, it needs a lot of expertise and time to replicate the mesh infrastructure which enables you to serve people from different geolocations with servers close to them and to really scale for very large sessions.

Like I mentioned earlier, you may go with a cloud provider for speed and reliability; see <https://mediasfu.com/pricing> (its mediasoup-based and will cost you way less than what AWS, Azure and the like will bill just based on bandwidth usage only).

5 [Reply](#) ...



Solid-Band3204 OP • 6mo ago

Man, thanks for your advice! I'm going to do the same thing for my use case using Redis. I totally get your point

(You make sure all 20 are assigned to the same server. Your task now is just keeping note of rooms available on specific servers and routing consumers there. That way, no consumer ends up on a server where the expected media isn't available)

I also love the product's pricing compared to LiveKit, which I think uses Pion under the hood. But right now, I want to build the thing on my own and at the same time, learn it in more depth.

1 [Reply](#) ...



msdosx86 • 6mo ago • Edited 6mo ago

Let's say it's allowed to have one room per SFU. So if room1 gets created in SFU1, all participants must connect to SFU1. This is an okay'ish approach if you're building something like Discord where (usually) a few people sit in a room (<25).

[+](#) Create

room1:192.168.0.15

Where 192.168.0.15 is a local IP address of the SFU. Next you need an orchestrator that stands before SFUs to decide to which SFU relay an incoming connection using the Redis record we previously stored.

So if Alice wanted to join to room1 the flow would be

Alice -> TURN (if NAT is strict) -> Orchestrator-> SFU1

Orchestrator also has to decide which SFU to pick (for new rooms) considering their CPU load so it's also a load balancer. So if you see that your single SFU1 is under heavy load you can spin up SFU2 and all new rooms will be connected to it.

2 [Reply](#) ...



Solid-Band3204 [OP](#) • 6mo ago

Cool, I'll definitely try this approach it's simple and straightforward for my use case. In my setup, I know how many peers each room contains, and I can decide how many rooms to allow per SFU based on my machine's capacity.

However, this approach breaks when I don't have control over how many peers can join a single room.

Thanks.

1 [Reply](#) ...



msdosx86 • 6mo ago

However, this approach breaks when I don't have control over how many peers can join a single room.

Wdym by that? You're using Mediasoup hence you're writing the room and the participants logic by yourself so you definitely have control over how many participants can join a single room.

1 [Reply](#) ...